Дисциплина «API-технологии»

Дополнительный материал к практической работе № 2

Как пользоваться OpenAPI и заполнять файл openapi.yaml

1. Для чего нужен этот материал

На практической работе № 2 вы проектируете API сервиса прогнозирования спроса или урожайности: определяете потребителя, ресурсы, запросы и ответы. Затем результат нужно оформить в файле openapi.yaml. Этот материал поможет сделать это без предварительного знакомства с OpenAPI. Работайте с уже согласованным на занятии проектом: придумывать новое API во время заполнения файла не требуется.

API — способ взаимодействия программ. OpenAPI — стандарт, по которому можно описать это взаимодействие: куда обратиться, что передать и какой ответ ожидать. openapi.yaml — обычный текстовый файл с таким описанием. Сам по себе файл не создаёт сервер, не обучает модель и не вычисляет прогноз. Это контракт, по которому в дальнейшем можно разрабатывать клиент, сервер и проверять соответствие реализации исходному проекту.

2. Что должно получиться

В результате у вас должен быть один файл openapi.yaml, отражающий именно ваш вариант практической работы № 2. Другой студент, не слышавший вашей защиты, должен понять по нему, как запросить прогноз, какие данные передать, что он получит при успехе и как выглядит ошибка. В файле должны согласовываться методы, пути, схемы входных и выходных данных, успешные и ошибочные ответы, ограничения и краткое правило совместимости.

Для домашнего задания используйте выданный преподавателем образец как основу. Если вы проектировали урожайность, замените товар, магазин и количество единиц товара на сущности своей предметной области. Не оставляйте в документе операции только потому, что они присутствовали в образце.

3. Как читать и заполнять openapi.yaml

У файла есть четыре основные части. Сначала прочитайте их назначение, затем переносите данные из заполненного бланка практической работы.

Раздел	**Что в нём указывается**	**Что сделать студенту**
openapi	Версия формата OpenAPI.	В выданном шаблоне оставьте 3.0.3.
info	Название, версия и	Назовите свой сервис и

	назначение вашего API.	опишите его назначение.
paths	Доступные пути, операции, входные данные и ответы.	Перенесите согласованные 3–4 операции.
components.schemas	Общие правила для объектов данных.	Опишите запрос, результат прогноза и ошибку.

Важно различать openapi: 3.0.3 — версию формата файла и info.version: 0.1.0 — версию вашего учебного API. Эти поля отвечают на разные вопросы.

Шаг 1. Уточните границы сервиса и заполните info.

Возьмите контекстную схему из ПР № 1 и таблицу операций из ПР № 2. Оставьте только маршруты, принадлежащие вашему сервису. Если история продаж хранится во внешней учётной системе, не выдавайте её интерфейс за маршрут собственного API. Затем запишите название API, его версию и краткое назначение в разделе info.

Пример. Сервис прогнозирования спроса создаёт прогноз по товару и периоду и позволяет менеджеру получить сохранённый результат. Историю продаж он берёт из другой системы, которая остаётся владельцем своего API.

Шаг 2. Перенесите согласованные операции в paths.

Для каждой строки таблицы операций укажите путь и HTTP-метод. Внутри операции запишите короткое назначение. Адрес /forecasts обозначает ресурс «прогнозы»; post на этом адресе описывает создание результата, get — получение списка. Адрес /forecasts/{forecast_id} обозначает один конкретный прогноз.

```
paths:
  /forecasts:
    post:
      summary: Создать прогноз
      responses:
        '201':
          description: Прогноз создан
```

Если в пути есть {forecast_id}, опишите параметр с name: forecast_id, in: path и required: true. Для строки запроса вроде /forecasts?product_id=12 нужен параметр in: query. При наличии JSON-тела для POST используйте requestBody. Не добавляйте POST, PUT или DELETE «для полноты», если они не нужны вашему сценарию.

Шаг 3. Опишите данные в components.schemas.

Схема — это не один пример прогноза, а правила для всех прогнозов: названия полей, их типы и обязательность. Например, product_id и store_id — целые числа, period — строка с месяцем прогноза, predicted_units — прогнозируемое количество единиц товара. В required

перечисляйте только обязательные поля. Для настоящего примера значений используйте вымышленные данные.

```
ForecastRequest:
  type: object
  required: [product_id, store_id, period]
  properties:
    product_id:
      type: integer
    store_id:
      type: integer
    period:
      type: string
      description: Месяц в формате YYYY-MM
```

Запись $ref: '#/components/schemas/ForecastRequest' — ссылка на схему внутри того же файла. Если переименуете схему, исправьте и все ссылки на неё.

Шаг 4. Покажите успешные и ошибочные ответы.

Каждой важной операции назначьте успешный ответ и нужные для вашего проекта ошибочные ответы. Например, после создания прогноза клиент получает код 201 и JSON с forecast_id и predicted_units; получение существующего прогноза возвращает 200. Если идентификатор не найден, предусмотрите 404 и понятное сообщение. Названия полей в схемах и примерах должны совпадать.

Пример. Для POST /forecasts клиент передаёт {"product_id": 12, "store_id": 3, "period": "2026-10"}. При успехе он получает код 201 и результат с forecast_id: 101 и predicted_units: 85. В учебном контракте отсутствие period можно обозначить кодом 400 и сообщением {"error": "period is required"}. Это выбранное правило проекта; будущий сервер должен быть реализован в соответствии с ним.

Шаг 5. Добавьте ограничения и правило совместимости.

Поясните неочевидные условия прямо в описаниях полей: например, period записывается как YYYY-MM, а predicted_units измеряется в единицах товара и не бывает отрицательным. В info.description кратко зафиксируйте правило совместимости: поля ответа, которые уже использует клиент, нельзя без согласования переименовывать или удалять. Так контракт остаётся понятным не только вам, но и разработчику клиентского приложения.

4. Как проверить файл перед сдачей

Сначала проверьте смысл: соответствует ли каждый маршрут таблице операций и потребителю из ПР № 1? Все ли ссылки $ref ведут к существующим схемам? Совпадают ли поля схем и примеров? Есть ли успешные и ошибочные ответы, обязательные поля, ограничения и правило совместимости?

Затем проверьте запись YAML. Для отступов используйте пробелы, а не табуляцию; элементы одного уровня выравнивайте одинаково. Коды ответов заключайте в кавычки (например, '201' и '404'), а период вида "2026-10" тоже удобно записывать в кавычках. Не исправляйте служебные имена полей по своему усмотрению: requestBody, responses и $ref должны быть написаны точно.

Если доступен Swagger Editor, вставьте в него содержимое файла: редактор поможет обнаружить ошибки записи и показать получающуюся документацию. Такая проверка подтверждает качество описания, но не означает, что ваш сервер уже существует или что запросы к нему работают.

5. На что обратить внимание

Не переписывайте учебный пример механически. Шаблон нужен для оформления вашего проекта, а не для замены результатов самостоятельного анализа. Например, если вы выбрали прогноз урожайности, результат не должен называться predicted_units товара без объяснения единиц измерения. Не смешивайте API разных систем и не оставляйте в файле противоречия между запросами, схемами и ответами.

На будущих занятиях, когда вы начнёте писать FastAPI-сервис, сравните его реальное поведение с этим контрактом. В частности, выбранная здесь ошибка с кодом 400 не появится автоматически только потому, что она записана в openapi.yaml: её потребуется согласовать с реализацией. Сейчас ваша задача — составить ясное и непротиворечивое описание, по которому эту реализацию можно будет сделать.